Hardware architecture

Data flow: the hardware first receives all weights and input samples, stores them in memory, runs the hidden-layer computation, then the output-layer computation, and finally sends the results back.

Control unit: an FSM manages operation in phases:
(1). Idle (2). read inputs (3). compute hidden layer (4). compute output layer (5). write outputs

Memory architecture: separate RAMs are used for different data types:
(1). input samples (2). hidden-layer weights (3). output-layer weights (4). hidden-layer results (5). final results

Hidden-layer hardware: reads the input data and two weight sets, then computes the two hidden neurons in parallel.

Activation function: uses a lookup-table for sigmoid implementation, which is hardware-friendly and simpler than computing the function directly.

Output-layer hardware: the final layer reuses the matrix-multiply ip from previous labs to combine hidden-layer outputs into the final classification results.

Trade-offs: It does not fully process everything in parallel (does not instantiate one multiplier/adder for every neuron and every sample at the same time). Instead, it reuses a smaller amount of hardware across multiple clock cycles to save resources. It uses a small compute block repeatedly over many clock cycles. So the tradeoff is lower hardware cost but longer computation time

Aspect	HDL Verilog version	HLS version	Effect on time
Top structure	FSM with stages: read, hidden, output, write	Streaming/dataflow style	HLS overlaps stages better
Hidden layer	Sequential control	Pipelined loop in mlp_hls.cpp	HLS reduces per-sample overhead
Memory use	Separate RAM modules with synchronous reads	Stream-based internal transfer	HDL pays extra read/write cycles
Parallelism	Limited, stage-by-stage	UNROLL on dot-product loops, DATAFLOW across stages	HLS has more parallel execution
Control overhead	Many state transitions and handshake waits	Less explicit FSM overhead in source code	HDL simulation shows more event activity
Comparison	Accurate hardware behavior	May be higher-level / less cycle-detailed	Can easily show 10x to 30x difference

Optimization cases in HLS

Modules Loops Hide empty columns Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
mlp_hls (8)	8.760E3		877		no	function	5	10	650	1,666	0
mlp_hls_Pipeline_VITIS_LOOP_64_1 (1)	660.000		65		loop auto-rew	function	0	0	9	56	0
mlp_hls_Pipeline_VITIS_LOOP_71_2 (1)	100.000		9		loop auto-rew	function	0	0	74	73	0
mlp_hls_Pipeline_VITIS_LOOP_76_3 (1)	100.000		9		loop auto-rew	function	0	0	74	73	0
mlp_hls_Pipeline_VITIS_LOOP_81_4 (1)	50.000		4		loop auto-rew	function	0	0	31	67	0
mlp_hls_Pipeline_VITIS_LOOP_86_5 (1)	4.500E3		449		loop auto-rew	function	0	0	50	182	0
mlp_hls_Pipeline_VITIS_LOOP_93_6 (1)	700.000		65		loop auto-rew	function	1	8	243	529	0
mlp_hls_Pipeline_VITIS_LOOP_111_8 (1)	700.000		65		loop auto-rew	function	0	2	113	138	0
mlp_hls_output	1.920E3	3	-	64	no	loop					

Case 1: no manual optimization

(1). total time is largest: 8760 ns; (2). read_X is the main slow part because the AXIS input comes in one word at a time;(3). mlp_hls_output is also slower because it is not pipelined;(4). Hardware cost is moderate: BRAM 5, DSP 10, FF 650, LUT 1666; here Vitis did some automatic loop pipelining, so this is not fully unoptimized

Modules Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
mlp_hls (8)	7.520E3		753		no	function	4	10	730	1,639	0
mlp_hls_Pipeline_init_bias (1)	660.000		65		loop auto-rew	function	0	0	9	56	0
mlp_hls_Pipeline_read_W1 (1)	100.000		9		loop auto-rew	function	0	0	74	73	0
mlp_hls_Pipeline_read_W2 (1)	100.000		9		loop auto-rew	function	0	0	74	73	0
mlp_hls_Pipeline_read_W3 (1)	50.000		4		loop auto-rew	function	0	0	31	67	0
mlp_hls_Pipeline_read_X (1)	4.500E3		449		loop auto-rew	function	0	0	40	273	0
mlp_hls_Pipeline_hidden_layer (1)	700.000		65		loop auto-rew	function	1	8	281	538	0
mlp_hls_Pipeline_output_layer (1)	700.000		65		loop auto-rew	function	0	2	137	139	0
mlp_hls_Pipeline_mlp_hls_output (1)	660.000		65		loop auto-rew	function	0	0	11	92	0

Case 2: pipeline + unroll + array partition/reshape

(1). The math parts run faster because some operations are done in parallel (2). Output writing also becomes faster, so total time drops to 7520 ns. (2). But improvement is moderate, because the biggest slow part is still read_X at 4500 ns. (3). So compute got faster, but input reading still limits the whole design. (4). Hardware cost changes only a little: BRAM 4, DSP 10, FF 730, LUT 1639. (5). This is a good balanced optimization: better speed, little extra hardware.

Modules Loops Hide empty columns Loops Hide empty columns

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
mlp_hls (4)	5.630E3		485		dataflow	function	1	10	3,161	3,361	0
Block_entry_S_AXIS_V_data_V_rd_proc (2)	4.840E3		484		no	function	0	0	432	969	0
hidden_layer (1)	700.000		64		loop auto-rew	function	1	8	412	584	0
output_layer (1)	700.000		64		loop auto-rew	function	0	2	125	209	0
write_results (1)	660.000		64		loop auto-rew	function	0	0	12	100	0

Case 3: dataflow + pipeline + unroll + array partition

(1). Fastest because stages can overlap: while one sample is being read, another can already be computed, and another can already be written. (2). Total time drops most, to 5630ns. (3). Big arrays were removed and replaced by streams, so BRAM drops a lot to 1. (4). control logic becomes much more complex, so FF and LUT jump a lot: FF 3161, LUT 3361. (5). This case gives the best speed, but uses much more logic.